

Inigo Quilez :: articles :: timing in ticks - 2017



Intro

When implementing animation systems or media editors, clock and timelines are one of those things that you have to be careful with. Common sense says you should express time with a high precision data type like `double` or `uint64_t` for your master clock and also for all your timestamps or timeline keys. Then, when computing your procedural animation or interpolate a property, just subtract the current clock's time from the timestamp/key, and **then** safely cast it to a less precise data type like `float` or `uint32_t` such that you can do all of your math at full speed without any resolution lost. This is similar to how you want to handle coordinate systems in game engines and rendering apps - express the transformation hierarchies in doubles and only cast to float after the `WorldToCamera` transformation.

This works great for procedural and sparsely sampled interpolation based animation, but things get a bit more complicated when doing frame-by-frame animation or video editing. Let me give you an example.

The problem

Consider a video recorded at 60 frames per second. Suppose this video is just a layer in a larger piece, that also contains hand drawn animations at 24 frames per second (the traditional rate at which film animators work). And let's say the output of this animated piece is on a VR headset that runs at 90 frames per second refresh rate. This was my life when I built the VR animation system [Quill](#), but any software that uses multiple (and recursive!) timelines and accepts different media types will be in a similar situation.

In my case, doing all the timeline management and media sampling with double precision microseconds time proved difficult. Regardless of rounding direction when sampling frames from a media file, there were always timelines not aligning properly due to some frames being repeated or skipped in different layers at different times. Switching to integers alone didn't help me, because at the end of the day the number 1,000,000,000 cannot be divided exactly by 24 exactly. Nor it can divide 60 or 90, so no matter what, I always had rounding

errors somewhere.

My solution

So in Quill I ended up using "ticks" for timing and animation instead of microseconds. It is a sort of "fixed point" precision approach, and it works this way: one second contains 25200 ticks. That allows it to operate exactly with lots of frame rates and refresh rates without ever mapping or repeating a frame, that is, without strobing. In particular, my "tick" is compatible with

- 24 hz and 48 hz, great for movie playback
- 6 hz, 8 hz and 12 hz, great for animating on 4s, 3s and 2s
- 30, 60, for internet video and TV in the USA
- 25 hz and 50 hz, for TV in the EU
- 72 hz, for oculus Quest 1
- 90 hz for Rift and other headsets
- 120, 144 and 240, for high frequency PC monitors
- and many more

There are many more expressible refresh rates, the full list of divisors of the number 25200 and hence all the frame rates that measuring in "ticks" can handle exactly is:

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 12, 14, 15, 16, 18, 20, 21, 24, 25, 28, 30, 35, 36, 40, 42, 45, 48, 50, 56, 60, 63, 70, 72, 75, 80, 84, 90, 100, 105, 112, 120, 126, 140, 144, 150, 168, 175, 180, 200, 210, 225, 240, 252, 280, 300, 315, 336, 350, 360, 400, 420, 450, 504, 525, 560, 600, 630, 700, 720, 840, 900, 1008, 1050, 1200, 1260, 1400, 1575, 1680, 1800, 2100, 2520, 2800, 3150, 3600, 4200, 5040, 6300, 8400, 12600, 25200

Compare that to the very limited ability of microseconds which has a lot fewer divisor and only allows for these framerates:

1, 2, 4, 5, 8, 10, 16, 20, 25, 32, 40, 50, 64, 80, 100, 125, 160, 200, 250, 320, 400, 500, 625, 800, 1000, 1250, 1600, 2000, 2500, 3125, 4000, 5000, 6250, 8000, 10000, 12500, 15625, 20000, 25000, 31250, 40000, 50000, 62500, 100000, 125000, 200000, 250000, 500000, 1000000

The list is even shorter for milliseconds and not even worth looking at.

The thing that "ticks" and hence Quill cannot handle is the NTSC frequencies of 59.94hz and 29.97hz typical in Japan, South Korea and the USA. This could be solved by making sure the "tick" contains the extra factors of 11 and 13, which together with the 7 that ticks already account for, can express the number 1001 ($7 \times 11 \times 13$), which is necessary for NTCS framerates.

Some implementation details

In terms of implementation, I measured ticks with data type `uint64_t`, which fits 23 million years worth of timeline. More importantly, I strongly typed `uint64_t` in a class that protected it from being used as an integer so that only valid type/framerate operations could be performed in it.

On Windows 10 and 11, my source of my "ticks" is the high frequency counter (RDTSC) in the system, but I think you could also use `QueryInterruptTimePrecise()` for more accuracy:

```
bool piTimer_Init( piTimer * me )
{
    LARGE_INTEGER li = { 0 };
    if( !QueryPerformanceFrequency(&li) )
        return false;
    me->pFreq = li.QuadPart;

    LARGE_INTEGER t;
    QueryPerformanceCounter(&t);
    me->hto = t.QuadPart;

    return true;
}

uint64_t piTimer_GetTicks( piTimer * m )
{
    LARGE_INTEGER t;
    QueryPerformanceCounter(&t);
    return (25200 * (t.QuadPart - me->hto)) / me->pFreq;
}
```

All the variables here are of type `uin64_t`. In my system `pFreq` is 10,000,000, so there's 3 orders of magnitude more precision that I need for individual ticks. Since 25,200 is about 23 bits, there are 41 bits left for the

multiplication, so we can safely run for 2^{41} seconds, which is 69 thousand years. If we want the full 23 million years range, we can cast the operands above to `uint128_t`, do the computation, then downcast the result to `uint64_t` before returning from `piTimer_GetTicks()`:

```
return (uint64_t)( (uint128_t(25200)
                  *uint128_t(t.QuadPart-me->hto))
                /uint128_t(me->pFreq));
```

inigo quilez - learning computer graphics since 1994